Time-space-head lower bounds for error-correcting-code encoders with sequential message access

Joshua Cook, Dana Moshkovitz

doi:10.4230/LIPIcs.CCC.2024.5; CC BY4.0

Allowed standard prerequisites (editorial).

- Pigeonhole principle, Markov inequality, union bound and restrictions of binary inputs. Author Lemmas52/53; complete local counting/averaging proof retained.

Author code, restriction and computation model definitions

Definition 8 (Code, Distance, and Rate). Let Σ_1 and Σ_2 be any alphabets over binary bits: $\Sigma_1 = \{0,1\}^a$ and $\Sigma_2 = \{0,1\}^b$ for some integers a and b. Then for any function $C : \Sigma_1^N \to \Sigma_2^M$, we say C is a code with relative distance δ if for any two $x, y \in \Sigma_1^N$ we have that $C(x)$ and $C(y)$ differ on at least δ fraction of indexes. We say that C has rate $\frac{N}{M}$. We say that an element $u \in \Sigma_2^M$ is a codeword of C if for some $x \in \Sigma_1^N$ we have that $C(x) = u$.

Definition 16 (Restriction). A restriction of length n is just a string $x^* \in \{0,1,*\}^n$. Any index where x^* is either 0 or 1 is fixed, and any index where x^* is not fixed, we say it is free. If x^* is a restriction with k free indexes, and there is a binary string $y \in \{0,1\}^k$, then we define the string $x_y = y \circ x^*$ to be the string that agrees with x^* on indexes where x^* is fixed, and on the jth index that x^* is free agrees with y_j.

Definition 6 (Sequential Oracle). Let x be some specific string of length n and h be an integer. Then a sequential oracle to x with h heads is a machine that has as state h integers within the range $[n]$. It has an input tape that can take up to two integers, $u_1, u_2 \in [h]$, indicating up to two of the h heads, and an operation which can be one of the following:

1. Move forward. This increments the u_1th head by one, if it is less than n.
2. Move backward. This decrements the u_1th head by one, if it is more than 1.

3. Read. Let i be the value of the u_1th head. Then this returns x_i.

4. Jump to. This sets the u_1th head to be the same value of the u_2th head.

Any one of these operations can be done in one time step. We call the oracle a non-reversible, sequential oracle if it can not use the "move backward" operation. We call the oracle a non-jumping, sequential oracle if it can not use the "jump to" operation. We say that an algorithm has sequential access to an input if the only way that input can be accessed is through a sequential oracle. We assume that all sequential oracles start with all heads at position 1.

Remark 44 (Space And State Of An Algorithm). To simplify our proof here, we will refer to the space of an algorithm as the size needed to hold its entire state, including:

- Its work tape.
- All head locations.
- Number of bits written.

We assume that an algorithm always prints its output bits in order, so the number of bits written is enough to know which bit will be output next. Space does not include the bits printed so far, or the input bits. If one interprets the space, S, to just be the size of the work tape, since we assumed $S \geq h \log(N)$, using our alternative definition of space only increases S by a constant factor, so our final results hold. So for simplicity, we assume S is the space needed to store its entire state.

Remark 45 (On Time And Uniformity). In this specific section on sequential lower bounds, we will consider a non-uniform model of computation. We allow the algorithm with sequential access to the input to use any function to define it's state transitions and head movements. The only condition is that such transitions are only functions of the working tape and whatever bits are under the heads to the input. In particular, the time in our algorithm lower bounds is actually the number of head movements.

Theorem 2 (Lower Bounds For Encoders With Sequential Access). Suppose C is a code with relative distance δ encoding N bits. Suppose A is an algorithm computing C running in time T space S and using h sequential heads to access the message. Further assume $S > h \log(N)$. Then

$$hST = \Omega(\delta N^2).$$

Our lower bounds work by partitioning the message into intervals and showing that most intervals need to be visited many times for the code to have good distance. This is because our space is bounded, so not much can be remembered about an interval when the heads leave it. So it must be visited many times for each of the different possible messages in that interval to have different things written to the code when no head is in that interval. Then the fact that our access to the message is sequential and we have few heads makes visiting an interval slow, requiring a long time to visit each interval enough times. First,

we need to formalize the idea that our algorithm can not have one of its heads enter new intervals in the message very often. But this is not true if we just count how many times the algorithm moves a head into an interval it was not in. As a counterexample, suppose a head is right next to the boundary of two intervals. Then the head can move in and out of it once every two time steps to visit it many times. In fact every interval may have been visited many times. So we need a more strict notion of visiting an interval. So instead, we want to only count the number of times an interval transitions from having no head near it (so in one of its neighboring intervals) to having a head in it. In this setup, our algorithm really has to spend a full intervals length worth of time transitioning from having every head far from an interval to having one inside it. So to formally describe this, we introduce interval marking, where we mark an interval when a head enters it, and only unmark it when all heads are far. It requires a lot of time to mark an interval after it has been unmarked. Before we define a marking, we need to define the distance of an interval to a head. This is defined in the obvious way.

Definition 46 (Distance). For any set $S \subseteq [N]$ and any head locations $H \subseteq [N]$ we define our distance between S and H as

$$\Delta(H, S) = \min_{h \in H, s \in S} |h - s|.$$

Now we can define our interval markings for an algorithm.

Definition 47 (Interval Marking). Let A be an algorithm running in time T and space S with sequential head access to the message and an interval length I. For a length N message x, let $m = \lceil \frac{N}{I} \rceil$. Partition the message into m length I intervals: $B_1, \ldots, B_m$ where $B_i = (I(i-1), Ii]$ with $B_m = (I(m-1), n]$. A marking of the intervals is just a set $a \in \{0,1\}^m$. For an algorithm A on a message x, for $t \in [T]$, let H^t be the set of intervals that A running on message x has a head in at time t. Then we inductively define a marking of A on message x. a^0 has nothing marked: $a^0 = 0^m$. At any subsequent time step t with head positions H, we define a^t by

$$a_i^t = \begin{cases} 1 & H^t \cap B_i \neq \varnothing \\ 0 & \Delta(H^t, B_i) \geq I \\ a_i^{t-1} & \text{otherwise.} \end{cases}$$

The sequence $a^0, \ldots, a^T$ is the I marking of A on message x. We say interval B_i is covered at time t during algorithm A if $a_i^t = 1$. We say A marks interval B_i at time t if $a_i^t = 1$ but $a_i^{t-1} = 0$ and A unmarks interval B_i at time t if $a_i^t = 0$ but $a_i^{t-1} = 1$. We say there is a marking at time t if A marks any interval at time t, and there was an unmarking at time t if A unmarks any interval at time t. We now emphasize, we use markings to refer to when an interval changes from uncovered to covered, and unmarking to when an interval changes from covered to uncovered. Marking always refers to this change, while cover always refers to how things are. For example, number of markings is how many times intervals change to be covered. Now using our terminology, we can formalize our argument. First, its straightforward to observe that if there are few heads, most intervals are uncovered as no heads are near them.

Lemma 48 (Max Number Of Covered Intervals). For any algorithm A running in time T, space S, and h heads, at any $t \in [T]$ the total number of intervals covered in an I marking of A on a length N message are at most $3h$.

Proof. See that if for any i, interval B_i is only covered if $\Delta(H, B_i) < I$. For any head h A has at time t, there are only at most 3 intervals that can be within I of h. Specifically, the one that h is in and the two next to it. Thus each individual head only covers at most 3 intervals, and there are only h heads, so only $3h$ intervals can be covered. □

Now we show that it takes a long time to mark many intervals.

Lemma 49 (Max Number of Markings/Unmarkings). For any algorithm A running in time T, space S, and h heads, any marking of A on a length N message makes at most $1 + T/I$ markings, and $1 + T/I$ unmarkings.

Proof. The idea is that after the first step and the first marking, each interval will take time I to get through to mark the next one. Then we only unmark an interval once it will take time I to mark it again. We only bound the number of markings, since the number of unmarkings is less than the number of markings. We formally bound the number of markings by showing each time step can only "contribute" to marking one interval, and that every marking of an interval needs at least I "contributions" every time it is marked. So we say any step t contributed to a marking of interval i if the head A moves at time t is in an interval adjacent to interval i and A moves that head toward interval i. We see that by this definition, A only contributes to one interval i per time step, since A can only move one head, and it either moves that head towards the interval above or below it. Let i_t be the interval that A contributes to at time t. Suppose at any time t, the head positions of A are H^t. Then for any interval B_i see that if $\Delta(H^t, B_i) \leq I$ and $\Delta(H^{t+1}, B_i) < \Delta(H^t, B_i)$ then it must be because $i_t = i$. Otherwise the closest head to B_i must not have moved toward it. Thus for the distance to decrease beyond I, it must be due to contributions from A. Further, if $\Delta(H^{t+1}, B_i) < \Delta(H^t, B_i)$, then $\Delta(H^{t+1}, B_i) = \Delta(H^t, B_i) - 1$ since heads can only move one position per time step. Thus to change $\Delta(H^t, B_i) \geq I$ to $\Delta(H^{t+t'}, B_i) = 0$ requires at least I contributions from A to B_i. Finally, see that after the first marking of the first interval that the distance of every uncovered interval to a head starts out at least I. And that every interval that is unmarked also starts with distance I from any head. Thus, after the first interval is marked, every new marking of an interval requires at least I contributions from A. And A can only contribute to one interval per time step. Thus A can only have at most $1 + T/I$ markings. Similarly, every unmarking must come from exactly one other marking, we also have at most $1 + T/I$ unmarkings. □

The argument becomes a bit more complex when we allow the algorithm to be adaptive. Specifically, which intervals are marked frequently may change depending on the message, as are the intervals that are uncovered frequently. Specifically, our adversarial inputs set everything outside of one interval to 0. If the algorithm knows that everything outside one interval will be zero, than it can detect which interval is non-zero and spend all its time on that interval. To get around these issues, we choose messages randomly. For any random message, for a random interval, with high probability that interval is only covered when a

small fraction of the output bits are written, and that interval is unmarked few times. Equivalently, we can select a random interval, randomly restrict everything outside that interval, than randomly assign that interval. These sample the same distribution, so we also have that for most random intervals chosen, B_i, and most restrictions outside B_i, it must be that for most assignments to B_i we have that B_i is unmarked few times and uncovered when most output bits are written. So we define a good restriction to be such a restriction, and then show that good restrictions give distinct messages that have too close code words. Finally we show good restrictions exist when $hST \ll \delta N^2$.

Definition 51 (A Good Restriction of the Message). For any algorithm A with time T and space S, given distance δ and interval length I, we say a restriction x^* is good for interval i with distance δ on algorithm A if the following holds.

1. x^* fixes every bit outside of the interval B_i, and leaves every bit inside B_i unfixed.
2. Let Y be the set of assignments for B_i such that for $x = y \circ x^*$ we have:
   (a) A on message x writes at most δ fraction of its bits when B_i is covered.
   (b) A on message x unmarks B_i at most $\frac{8T}{N}$ times.

   We also require that $|Y| > 2^I/2$.

Now we show that a good restriction is enough to give our lower bounds.

Lemma 52 (Good Restrictions Give Lower Bounds). Suppose A is an algorithm with time T, space S, and h sequential heads to the message. If x^* is good for interval i with interval size I and distance δ, then whatever code A outputs has distance at most δ if

$$8ST < N(I-1).$$

Proof. The idea is just to consider the Y from Definition 51. For the x that come from Y, we have that B_i is unmarked rarely, so rarely in fact that multiple x must have the exact same states every time B_i is unmarked. Since the state includes the positions of the heads, they must write the same thing when the interval is uncovered. Since the state includes the number of bits written, they must be written at the same location when B_i is uncovered. Thus the distance between these messages is at most what is written when B_i is covered, which is at most a δ fraction of bits. More rigorously, for any $y \in Y$, let $x_y = y \circ x^*$. By definition, A on message x_y marks B_i at most $\frac{8T}{N}$ times. Then define $R(y)$ to be the tuple of the state of A at every step B_i is uncovered when running on message x_y. Then since the state only has S bits and B_i is uncovered at most $\frac{8T}{N}$ times, we have

$$|R(y)| \leq S\frac{8T}{N}.$$

Now see that since $|R(y)| \leq 8\frac{ST}{N}$, then the total number of distinct values for $R(y)$ is at most $2^{8\frac{ST}{N}}$. Since the restriction is good, the total number of distinct

$y \in Y$ is at least $2^I/2$. Finally, by lemma premise, we have that $8ST < N(I-1)$. Thus we can show that

$$\begin{aligned} 8ST &< N(I-1) \\ 8\frac{ST}{N} &< I-1 \\ 2^{8S\frac{T}{N}} &< 2^I/2 \end{aligned}$$

Thus the number of distinct $R(y)$ is less than the number of distinct y, so by pigeonhole principle, there must be $y_1, y_2 \in Y$ such that $R(y_1) = R(y_2)$. Now by definition of $R(y)$, when running A on message x_y we must have that everything written when interval B_i is uncovered is dependent only on $R(y)$ and x^*. In particular, $R(y_1)$ and $R(y_2)$ write the same bits when B_i is uncovered, at the same places. Since $y_1, y_2 \in Y$, we have that at most δ fraction of the bits are written when B_i is covered, and these are the only bits where they can differ. So the distance between the outputs of A on x_{y_1} and x_{y_2} is at most δ. □

It remains to show that there must be some good restriction if I is chosen well. Specifically when $I < \frac{N\delta}{24h}$.

Lemma 53 (Good Restrictions Exist). Suppose A is an algorithm with time T, space S, and h sequential heads to access the message. Then if $I < \frac{N\delta}{24h} < N < T$ then there is some restriction which is good for some interval i with distance δ on algorithm A.

Proof. Suppose by way of contradiction that every restriction is not good for any interval with distance δ on algorithm A. The idea is to show that, in expectation, either more than $3h$ of intervals are covered whenever a bit is written (contradicting Lemma 48) or more than $1 + T/I$ intervals are marked (contradicting Lemma 49). So choose a random interval, i, a random restriction x^* for every variable outside B_i. We assumed that x^* is bad, so with probability at least $1/2$ for a random assignment, y, of the variables in B_i will $y \notin Y$ for the Y defined in Definition 51. That is, with probability at least $1/2$ will we have for $x_y = y \circ x^*$ that algorithm A on message x_y will unmark interval i more then $\frac{8T}{N}$ times or will be covered when at least δ fraction of the output bits written. See that x_y is uniformly randomly distributed and so is i, and these are independent of each other. Thus in expectation over a randomly chosen interval, B_i, and a randomly chosen input, x, we have that with probability at least $1/2$ algorithm A on input x either unmarks interval B_i more then $\frac{8T}{N}$ times or interval B_i be covered when at least a δ fraction of the output bits written. Now we show that x cannot have enough intervals covered or unmarked to achieve this. By Lemma 49, the total number of markings is at most $T/I+1 < 2T/I$. So at most $\frac{N}{4I}$ intervals are marked more than $\frac{8T}{N}$ times. There are at least N/I intervals, so only at most $\frac{1}{4}$ fraction of the intervals can be marked more than $\frac{8T}{N}$ times. By Lemma 48, at any given time, at most $3h$ intervals can be covered. So the

probability that a random interval is covered is at most

$$\begin{aligned}\frac{3h}{N/I} &= \frac{3hI}{N} \\ &< \frac{3hN\delta}{N24h} \\ &= \frac{\delta}{8}.\end{aligned}$$

Then by a Markov inequality, the probability that a random interval is covered greater than a δ fraction of the times an output bit is written is at most $\frac{1}{8}$. Thus by a union bound, the total of fraction of intervals that are either unmarked more than $\frac{8T}{N}$ times or covered for at least δ fraction of the times an output bit is written is $\frac{3}{8} < \frac{1}{2}$. But by choice of x, these must occur for at least half i. Contradiction. So some restriction and interval must be good with distance δ. □

Now that we know good restrictions exist, and good restrictions imply our lower bounds, we can prove Theorem 2.

Proof of Theorem 2. Let $I = \frac{N\delta}{50h} < \frac{N(\delta/2)}{24h}$. Then by Lemma 53, a good restriction with distance $\delta/2$ exists. Then by Lemma 52, since C has distance greater than $\delta/2$, it must be the case that $32ST \geq NI$. Thus

$$\begin{aligned}8ST &\geq N(I-1) \\ 16ST &\geq NI \\ &= N\frac{N\delta}{50h} \\ 800STh &\geq N^2\delta \\ hST &= \Omega(n^2\delta).\end{aligned}$$

□

References

[1] Karl Abrahamson. Time-space tradeoffs for algebraic problems on general sequential machines. Journal of Computer and System Sciences, 43(2):269–289, 1991. doi:10.1016/0022-0000(91) 90014-V.

[2] N. Alon, J. Bruck, J. Naor, M. Naor, and R.M. Roth. Construction of asymptotically good low- rate error-correcting codes through pseudo-random graphs. IEEE Transactions on Information Theory, 38(2):509–516, 1992. doi:10.1109/18.119713.

[3] Laasya Bangalore, Rishabh Bhadauria, Carmit Hazay, and Muthuramakrishnan Venkitas- ubramaniam. On black-box constructions of time and space efficient sublinear arguments from symmetric-key primitives. In Theory of Cryptography: 20th International Conference, TCC 2022, Chicago, IL, USA, November 7–10, 2022, Proceedings, Part I, pages 417–446. Springer-Verlag, 2022. doi:10.1007/978-3-031-22318-1_15.

[4] L.M.J. Bazzi and S.K. Mitter. Endcoding complexity versus minimum distance. IEEE Transactions on Information Theory, 51(6):2103–2112, 2005. doi:10.1109/TIT.2005.847727.

[5] P. Beame. A general sequential time-space tradeoff for finding unique elements. In Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing, STOC '89, pages 197–203, New York, NY, USA, 1989. Association for Computing Machinery. doi:10.1145/73007.73026.

[6] Paul Beame and Niels Kornerup. Cumulative memory lower bounds for randomized and quantum computation. In Kousha Etessami, Uriel Feige, and Gabriele Puppis, editors, 50th International Colloquium on Automata, Languages, and Programming (ICALP 2023), volume

[7] of Leibniz International Proceedings in Informatics (LIPIcs), pages 17:1–17:20, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs. ICALP.2023.17.

[8] Paul Beame, Michael Saks, Xiaodong Sun, and Erik Vee. Time-space tradeoff lower bounds for randomized computation of decision problems. J. ACM, 50(2):154–195, 2003. doi: 10.1145/636865.636867.

[9] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, and Eran Tromer. On the concrete efficiency of probabilistically-checkable proofs. In Proceedings of the Forty-Fifth Annual ACM Symposium on Theory of Computing, STOC '13, pages 585–594. Association for Computing Machinery, 2013. doi:10.1145/2488608.2488681.

[10] Charles H. Bennett. Time/space trade-offs for reversible computation. SIAM J. Comput., 18(4):766–776, 1989. doi:10.1137/0218053.

[11] Nir Bitansky and Alessandro Chiesa. Succinct arguments from multi-prover interactive proofs and their efficiency benefits. In Proceedings of the 32nd Annual Cryptology Conference on Advances in Cryptology — CRYPTO 2012 - Volume 7417, pages 255–272, Berlin, Heidelberg, 2012. Springer-Verlag. doi:10.1007/978-3-642-32009-5_16.

[12] A. Borodin and S. Cook. A time-space tradeoff for sorting on a general sequential model of computation. SIAM J. Comput., 11(2):287–297, 1982. doi:10.1137/0211022.

[13] M. Capalbo, O. Reingold, S. Vadhan, and A. Wigderson. Randomness conductors and constant- degree lossless expanders. In Proceedings 17th IEEE Annual Conference on Computational Complexity, pages 8–8, 2002. doi:10.1109/CCC.2002.1004327.

[14] Eshan Chattopadhyay, Vipul Goyal, and Xin Li. Non-malleable extractors and codes, with their many tampered extensions. In Proceedings of the Forty-Eighth Annual ACM Symposium on Theory of Computing, STOC '16, pages 285–298. Association for Computing Machinery, 2016. doi:10.1145/2897518.2897547.

[15] M. Cheraghchi, F. Didier, and A. Shokrollahi. Invertible extractors and wiretap protocols. IEEE Trans. Inf. Theor., 58(2):1254–1274, 2012. doi:10.1109/TIT.2011.2170660.

[16] Mahdi Cheraghchi and Venkatesan Guruswami. Non-malleable coding against bit-wise and split-state tampering. J. Cryptol., 30(1):191–241, 2017. doi:10.1007/s00145-015-9219-z.

[17] Joshua Cook and Dana Moshkovitz. Time and space efficient deterministic decoders. Elec- tronic Colloquium on Computational Complexity, 2024. URL: https://eccc.weizmann.ac. il/report/2024/110/.

[18] Yotam Dikstein, Irit Dinur, and Shiri Sivan. The linear time encoding scheme fails to encode, 2023. arXiv:2312.16125.

[19] Irit Dinur, Shai Evra, Ron Livne, Alexander Lubotzky, and Shahar Mozes. Locally testable codes with constant rate, distance, and locality. In Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2022, pages 357–374. Association for Computing Machinery, 2022. doi:10.1145/3519935.3520024.

[20] Dariush Divsalar, Hui Jin, and Robert J. McEliece. Coding theorems for 'turbo-like' codes. In Proceedings 36th Annual Allerton Conference on Communication, Control, and Computing, pages 201–210, 1998. URL: https://api.semanticscholar.org/CorpusID:1045655.

[21] R. Gallager. Low-density parity-check codes. IRE Transactions on Information Theory, 8(1):21–28, 1962. doi:10.1109/TIT.1962.1057683.

[22] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating computation: Interactive proofs for muggles. J. ACM, 62(4), September 2015. doi:10.1145/2699436.

[23] André Gronemeier. A note on the decoding complexity of error-correcting codes. Inf. Process. Lett., 100(3):116–119, 2006. doi:10.1016/j.ipl.2006.06.006.

[24] V. Guruswami and P. Indyk. Linear-time encodable/decodable codes with near-optimal rate. IEEE Transactions on Information Theory, 51(10):3393–3400, 2005. doi:10.1109/TIT.2005. 855587.

[25] Venkatesan Guruswami and Widad Machmouchi. Explicit interleavers for a repeat accumulate accumulate (raa) code construction. In 2008 IEEE International Symposium on Information Theory, pages 1968–1972, 2008. doi:10.1109/ISIT.2008.4595333.

[26] Venkatesan Guruswami, Christopher Umans, and Salil Vadhan. Unbalanced expanders and randomness extractors from parvaresh-vardy codes. In Twenty-Second Annual IEEE Conference on Computational Complexity (CCC'07), pages 96–108, 2007. doi:10.1109/CCC.2007.38.

[27] Anna Gál, Kristoffer Arnsfelt Hansen, Michal Koucký, Pavel Pudlák, and Emanuele Viola. Tight bounds on computing error-correcting codes by bounded-depth circuits with arbitrary gates. IEEE Transactions on Information Theory, 59(10):6611–6627, 2013. doi:10.1109/TIT. 2013.2270275.

[28] R. W. Hamming. Error detecting and error correcting codes. The Bell System Technical Journal, 29(2):147–160, 1950. doi:10.1002/j.1538-7305.1950.tb00463.x.

[29] Tzvika Hartman and Ran Raz. On the distribution of the number of roots of polynomials and explicit weak designs. Random Struct. Algorithms, 23(3):235–263, October 2003. doi: 10.1002/rsa.10095.

[30] F.C. Hennie. One-tape, off-line turing machine computations. Information and Control, 8(6):553–578, 1965. doi:10.1016/S0019-9958(65)90399-2.

[31] Justin Holmgren and Ron Rothblum. Delegating computations with (almost) minimal time and space overhead. In 2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS), pages 124–135, 2018. doi:10.1109/FOCS.2018.00021.

[32] R. Impagliazzo, L. A. Levin, and M. Luby. Pseudo-random generation from one-way functions. In Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing, STOC '89, pages 12–24. Association for Computing Machinery, 1989. doi:10.1145/73007.73009.

[33] S. Jukna. A nondeterministic space-time tradeoff for linear codes. Information Processing Letters, 109(5):286–289, 2009. doi:10.1016/j.ipl.2008.11.001.

[34] Itay Kalev and Amnon Ta-Shma. Unbalanced Expanders from Multiplicity Codes. In Amit Chakrabarti and Chaitanya Swamy, editors, Approximation, Randomization, and Combin- atorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2022), volume 245 of Leibniz International Proceedings in Informatics (LIPIcs), pages 12:1–12:14, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs. APPROX/RANDOM.2022.12.

[35] Swastik Kopparty, Or Meir, Noga Ron-Zewi, and Shubhangi Saraf. High-rate locally-correctable and locally-testable codes with sub-polynomial query complexity. In Proceedings of the Forty-Eighth Annual ACM Symposium on Theory of Computing, STOC '16, pages 202–215. Association for Computing Machinery, 2016. doi:10.1145/2897518.2897523.

[36] Swastik Kopparty, Shubhangi Saraf, and Sergey Yekhanin. High-rate codes with sublinear-time decoding. J. ACM, 61(5), 2014. doi:10.1145/2629416.

[37] S. Rao Kosaraju. Real-time simulation of concatenable double-ended queues by double-ended queues (preliminary version). In Proceedings of the Eleventh Annual ACM Symposium on Theory of Computing, STOC '79, pages 346–351. Association for Computing Machinery, 1979. doi:10.1145/800135.804427.

[38] Xin Li. Improved non-malleable extractors, non-malleable codes and independent source extractors. In Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2017, pages 1144–1156. Association for Computing Machinery, 2017. doi: 10.1145/3055399.3055486.

[39] D. J. C. Mackay and Radford M. Neal. Near shannon limit performance of low density parity check codes. Electronics Letters, 33:457–458, 1996. URL: https://api.semanticscholar. org/CorpusID:122801915.

[40] Noam Nisan and David Zuckerman. Randomness is linear in space. J. Comput. Syst. Sci., 52(1):43–52, 1996. doi:10.1006/jcss.1996.0004.

[41] Wolfgang J. Paul, Joel I. Seiferas, and Janos Simon. An information-theoretic approach to time bounds for on-line computation (preliminary version). In Proceedings of the Twelfth Annual ACM Symposium on Theory of Computing, STOC '80, pages 357–367. Association for Computing Machinery, 1980. doi:10.1145/800141.804685.

[42] Ran Raz, Omer Reingold, and Salil Vadhan. Extracting all the randomness and reducing the error in trevisan's extractors. In Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing, STOC '99, pages 149–158. Association for Computing Machinery, 1999. doi:10.1145/301250.301292.

[43] I.S. Reed. A brief history of the development of error correcting codes. Computers & Mathematics with Applications, 39(11):89–93, 2000. doi:10.1016/S0898-1221(00)00112-7.

[44] O. Reingold, R. Shaltiel, and A. Wigderson. Extracting randomness via repeated condensing. In Proceedings 41st Annual Symposium on Foundations of Computer Science, pages 22–31, 2000. doi:10.1109/SFCS.2000.892008.

[45] Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. Constant-round interactive proofs for delegating computation. In Proceedings of the Forty-Eighth Annual ACM Symposium on Theory of Computing, STOC '16, pages 49–62. Association for Computing Machinery, 2016. doi:10.1145/2897518.2897652.

[46] Nandakishore Santhi and Alexander Vardy. Minimum distance of codes and their branching program complexity. In 2006 IEEE International Symposium on Information Theory, pages 1490–1494, 2006. doi:10.1109/ISIT.2006.262116.

[47] Walter J. Savitch and Paul M. B. Vitányi. Linear time simulation of multihead turing machines with head-to-head jumps. In Proceedings of the Fourth Colloquium on Automata, Languages and Programming, pages 453–464, Berlin, Heidelberg, 1977. Springer-Verlag.

[48] M. Sipser and D.A. Spielman. Expander codes. IEEE Transactions on Information Theory, 42(6):1710–1722, 1996. doi:10.1109/18.556667.

[49] D.A. Spielman. Linear-time encodable and decodable error-correcting codes. IEEE Transactions on Information Theory, 42(6):1723–1731, 1996. doi:10.1109/18.556668.

[50] Daniel A. Spielman. Linear-time encodable and decodable error-correcting codes. In Proceedings of the Twenty-Seventh Annual ACM Symposium on Theory of Computing, STOC '95, pages 388–397. Association for Computing Machinery, 1995. doi:10.1145/225058.225165.

[51] Amnon Ta-Shma, Christopher Umans, and David Zuckerman. Loss-less condensers, unbalanced expanders, and extractors. In Proceedings of the Thirty-Third Annual ACM Symposium on Theory of Computing, STOC '01, pages 143–152. Association for Computing Machinery, 2001. doi:10.1145/380752.380790.

[52] Luca Trevisan. Construction of extractors using pseudo-random generators (extended abstract). In Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing, STOC '99, pages 141–148. Association for Computing Machinery, 1999. doi:10.1145/301250.301289.

[53] Yaacov Yesha. Time-space tradeoffs for matrix multiplication and the discrete fourier transform on any general sequential random-access computer. Journal of Computer and System Sciences, 29(2):183–197, 1984. doi:10.1016/0022-0000(84)90029-1.